

Resumen Completo - Comandos SQL para Examen

DDL - Data Definition Language (Definición de Datos)

CREATE TABLE - Crear Tablas

Sintaxis básica:

```
CREATE TABLE nombre_tabla (  
    campo1 TIPO restricciones,  
    campo2 TIPO restricciones,  
    ...  
    PRIMARY KEY (campo),  
    FOREIGN KEY (campo) REFERENCES otra_tabla(campo) [ON DELETE accion] [ON UPDATE a  
);
```

Tipos de datos más comunes:

- INT / INTEGER - Números enteros
- VARCHAR(n) - Cadenas de texto variable hasta n caracteres
- CHAR(n) - Cadenas de texto fija de n caracteres
- TEXT - Texto largo
- DATE - Fechas (YYYY-MM-DD)
- DATETIME - Fecha y hora (YYYY-MM-DD HH:MM:SS)
- DECIMAL(p, s) - Números decimales (p=total dígitos, s=decimales)
- BOOLEAN / TINYINT(1) - Verdadero/Falso

Restricciones principales:

- NOT NULL - No puede ser nulo
- UNIQUE - Valor único en la tabla
- PRIMARY KEY - Clave primaria (automáticamente NOT NULL y UNIQUE)
- AUTO_INCREMENT - Incremento automático (solo para números)

- `DEFAULT valor` - Valor por defecto

Acciones de integridad referencial:

- `ON DELETE CASCADE` - Elimina registros relacionados
- `ON DELETE SET NULL` - Pone NULL en registros relacionados
- `ON DELETE RESTRICT` - Impide eliminación si hay registros relacionados
- `ON UPDATE CASCADE` - Actualiza registros relacionados

Ejemplo completo:

```
CREATE TABLE productos (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(100) NOT NULL UNIQUE,  
    precio DECIMAL(10,2) NOT NULL DEFAULT 0.00,  
    categoria_id INT,  
    fecha_creacion DATE DEFAULT (CURDATE()),  
    FOREIGN KEY (categoria_id) REFERENCES categorias(id) ON DELETE SET NULL  
);
```

ALTER TABLE - Modificar Tablas

Añadir columnas:

```
ALTER TABLE nombre_tabla ADD COLUMN nuevo_campo TIPO restricciones;
```

Modificar columnas existentes:

```
ALTER TABLE nombre_tabla MODIFY COLUMN campo NUEVO_TIPO nuevas_restricciones;
```

Eliminar columnas:

```
ALTER TABLE nombre_tabla DROP COLUMN campo;
```

Añadir restricciones:

```
-- Clave primaria
```

```
ALTER TABLE nombre_tabla ADD PRIMARY KEY (campo);
```

```
-- Clave foránea
```

```
ALTER TABLE nombre_tabla ADD FOREIGN KEY (campo) REFERENCES otra_tabla(campo);
```

```
-- Restricción única
```

```
ALTER TABLE nombre_tabla ADD UNIQUE (campo);
```

Eliminar restricciones:

```
ALTER TABLE nombre_tabla DROP FOREIGN KEY nombre_restriccion;
```

```
ALTER TABLE nombre_tabla DROP PRIMARY KEY;
```

CREATE INDEX - Crear Índices

Sintaxis:

```
CREATE INDEX nombre_indice ON tabla(campo1, campo2, ...);
```

```
CREATE UNIQUE INDEX nombre_indice ON tabla(campo);
```

¿Cuándo usar índices?

- Campos que se consultan frecuentemente en WHERE
- Campos usados en JOIN
- Campos usados en ORDER BY
- **NO crear índices** en tablas pequeñas o campos que cambian mucho

Ejemplo:

```
CREATE INDEX idx_apellido ON empleados(apellido);
```

```
CREATE INDEX idx_fecha_nombre ON pedidos(fecha, cliente_nombre);
```

DROP - Eliminar Estructuras

```
DROP TABLE nombre_tabla; -- Elimina tabla completa
```

```
DROP INDEX nombre_indice ON tabla; -- Elimina índice
```

```
DROP PROCEDURE nombre_procedimiento; -- Elimina procedimiento
DROP TRIGGER nombre_trigger;         -- Elimina trigger
```

DQL - Data Query Language (Consultas)

SELECT - Consultas Básicas

Sintaxis completa:

```
SELECT [DISTINCT] campos
FROM tabla1 [alias1]
[JOIN tabla2 [alias2] ON condicion]
[WHERE condiciones]
[GROUP BY campos]
[HAVING condiciones_de_grupo]
[ORDER BY campos [ASC|DESC]]
[LIMIT numero [OFFSET numero]];
```

Operadores en WHERE:

- =, !=, <>, <, >, <=, >= - Comparación
- LIKE 'patrón' - Búsqueda de patrones (% = cualquier cadena, _ = un carácter)
- IN (valor1, valor2, ...) - Está en la lista
- BETWEEN valor1 AND valor2 - Entre dos valores
- IS NULL / IS NOT NULL - Valores nulos
- AND, OR, NOT - Operadores lógicos

Ejemplos:

```
-- Búsqueda con comodines
SELECT * FROM empleados WHERE nombre LIKE 'Juan%';
SELECT * FROM productos WHERE codigo LIKE '_A_B%';

-- Rangos y listas
SELECT * FROM productos WHERE precio BETWEEN 100 AND 500;
SELECT * FROM empleados WHERE departamento IN ('Ventas', 'Marketing');

-- Valores nulos
SELECT * FROM clientes WHERE telefono IS NOT NULL;
```

JOIN - Unir Tablas

INNER JOIN - Solo registros que coinciden en ambas tablas:

```
SELECT campos
FROM tabla1 t1
INNER JOIN tabla2 t2 ON t1.campo = t2.campo;
```

LEFT JOIN - Todos los registros de la tabla izquierda:

```
SELECT campos
FROM tabla1 t1
LEFT JOIN tabla2 t2 ON t1.campo = t2.campo;
```

RIGHT JOIN - Todos los registros de la tabla derecha:

```
SELECT campos
FROM tabla1 t1
RIGHT JOIN tabla2 t2 ON t1.campo = t2.campo;
```

FULL OUTER JOIN - Todos los registros de ambas tablas:

```
SELECT campos
FROM tabla1 t1
FULL OUTER JOIN tabla2 t2 ON t1.campo = t2.campo;
```

Cuándo usar cada JOIN:

- **INNER JOIN:** Solo quiero datos que existen en ambas tablas
- **LEFT JOIN:** Quiero todos los registros de la primera tabla, aunque no tengan correspondencia
- **RIGHT JOIN:** Quiero todos los registros de la segunda tabla
- **FULL OUTER JOIN:** Quiero todos los registros de ambas tablas

Ejemplo práctico:

```
-- Clientes con sus pedidos (solo clientes que han pedido)
SELECT c.nombre, p.fecha, p.total
FROM clientes c
INNER JOIN pedidos p ON c.id = p.cliente_id;

-- Todos los clientes, aunque no hayan pedido nada
SELECT c.nombre, p.fecha, p.total
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id;
```

Funciones de Agregación

Funciones principales:

- COUNT(*) - Cuenta todas las filas
- COUNT(campo) - Cuenta filas no nulas del campo
- SUM(campo) - Suma valores numéricos
- AVG(campo) - Promedio de valores numéricos
- MAX(campo) - Valor máximo
- MIN(campo) - Valor mínimo

GROUP BY - Agrupar resultados:

```
SELECT campo_agrupacion, COUNT(*), SUM(campo)
FROM tabla
GROUP BY campo_agrupacion;
```

HAVING - Filtrar grupos:

```
SELECT departamento, COUNT(*) as total
FROM empleados
GROUP BY departamento
HAVING COUNT(*) > 5; -- Solo departamentos con más de 5 empleados
```

Diferencia WHERE vs HAVING:

- WHERE filtra filas **antes** de agrupar
- HAVING filtra grupos **después** de agrupar

Ejemplo completo:

```
SELECT
    categoria,
    COUNT(*) as total_productos,
    AVG(precio) as precio_promedio,
    MAX(precio) as precio_maximo
FROM productos
WHERE precio > 0  -- Filtrar productos con precio
GROUP BY categoria
HAVING COUNT(*) >= 3  -- Solo categorías con 3+ productos
ORDER BY precio_promedio DESC;
```

Subconsultas

Subconsulta escalar (devuelve un valor):

```
SELECT nombre, salario
FROM empleados
WHERE salario > (SELECT AVG(salario) FROM empleados);
```

Subconsulta con IN:

```
SELECT nombre
FROM empleados
WHERE departamento_id IN (
    SELECT id FROM departamentos WHERE ubicacion = 'Madrid'
);
```

Subconsulta con EXISTS:

```
SELECT c.nombre
FROM clientes c
WHERE EXISTS (
    SELECT 1 FROM pedidos p WHERE p.cliente_id = c.id
);
```



DML - Data Manipulation Language (Manipulación de Datos)

INSERT - Insertar Datos

Insertar valores específicos:

```
INSERT INTO tabla (campo1, campo2, campo3)
VALUES (valor1, valor2, valor3);
```

-- Múltiples filas

```
INSERT INTO tabla (campo1, campo2)
VALUES
    (valor1a, valor2a),
    (valor1b, valor2b),
    (valor1c, valor2c);
```

Insertar desde otra consulta:

```
INSERT INTO tabla_destino (campo1, campo2)
SELECT campo1, campo2
FROM tabla_origen
WHERE condicion;
```

Funciones útiles para INSERT:

```
INSERT INTO empleados (nombre, fecha_contrato, salario)
VALUES
    ('Juan Pérez', CURDATE(), 2500.00),
    ('Ana García', NOW(), 2800.00);
```

UPDATE - Actualizar Datos

Sintaxis:

```
UPDATE tabla
SET campo1 = nuevo_valor1,
    campo2 = nuevo_valor2
WHERE condiciones;
```


Ejemplos:

```
-- Actualización simple
```

```
UPDATE empleados  
SET salario = 3000  
WHERE id = 123;
```

```
-- Actualización con cálculo
```

```
UPDATE productos  
SET precio = precio * 1.10 -- Incremento del 10%  
WHERE categoria = 'Electrónicos';
```

```
-- Actualización con subconsulta
```

```
UPDATE empleados  
SET salario = (  
    SELECT AVG(salario) FROM empleados e2 WHERE e2.departamento = empleados.departamento  
)  
WHERE categoria = 'Junior';
```

⚠ IMPORTANTE: Siempre usar WHERE en UPDATE, o se actualizarán TODOS los registros.

DELETE - Eliminar Datos

Sintaxis:

```
DELETE FROM tabla  
WHERE condiciones;
```

Ejemplos:

```
-- Eliminar registros específicos
```

```
DELETE FROM empleados  
WHERE fecha_baja IS NOT NULL;
```

```
-- Eliminar con subconsulta
```

```
DELETE FROM productos  
WHERE categoria_id IN (  
    SELECT id FROM categorias WHERE activa = 0  
);
```

⚠ **IMPORTANTE:** Siempre usar WHERE en DELETE, o se eliminarán TODOS los registros.

Procedimientos Almacenados

Sintaxis Básica

```
DELIMITER $$

CREATE PROCEDURE nombre_procedimiento(
    IN parametro_entrada TIPO,
    OUT parametro_salida TIPO,
    INOUT parametro_entrada_salida TIPO
)
BEGIN
    -- Declaración de variables
    DECLARE variable_local TIPO DEFAULT valor;

    -- Lógica del procedimiento

END$$

DELIMITER ;
```

Tipos de Parámetros

- IN - Solo entrada, no se modifica
- OUT - Solo salida, se asigna dentro del procedimiento
- INOUT - Entrada y salida, se puede leer y modificar

Variables y Declaraciones

```
-- Declarar variables
DECLARE contador INT DEFAULT 0;
DECLARE nombre_completo VARCHAR(100);
DECLARE fecha_actual DATE;

-- Asignar valores
SET contador = 10;
SET fecha_actual = CURDATE();
```

```
-- Asignar desde consulta
SELECT COUNT(*) INTO contador FROM empleados;
SELECT CONCAT(nombre, ' ', apellido) INTO nombre_completo
FROM empleados WHERE id = 123;
```

Estructuras de Control

Condicional IF:

```
IF condicion THEN
    -- Acciones si es verdadero
ELSEIF otra_condicion THEN
    -- Acciones si es la segunda condición
ELSE
    -- Acciones si ninguna condición es verdadera
END IF;
```

Bucle WHILE:

```
WHILE condicion DO
    -- Acciones del bucle
    SET contador = contador + 1;
END WHILE;
```

Bucle FOR (con cursor):

```
DECLARE done INT DEFAULT FALSE;
DECLARE cur CURSOR FOR SELECT campo FROM tabla;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

OPEN cur;
read_loop: LOOP
    FETCH cur INTO variable;
    IF done THEN
        LEAVE read_loop;
    END IF;
    -- Procesar variable
END LOOP;
CLOSE cur;
```

Manejo de Errores

```
-- Lanzar error personalizado
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error personalizado';

-- Capturar errores
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
BEGIN
    -- Acciones cuando hay error
    ROLLBACK;
END;
```

Ejemplo Completo de Procedimiento

```
DELIMITER $$

CREATE PROCEDURE ActualizarSalarioEmpleado(
    IN emp_id INT,
    IN porcentaje_aumento DECIMAL(5,2),
    OUT mensaje VARCHAR(200)
)
BEGIN
    DECLARE salario_actual DECIMAL(10,2);
    DECLARE empleado_existe INT DEFAULT 0;

    -- Verificar que el empleado existe
    SELECT COUNT(*) INTO empleado_existe
    FROM empleados WHERE id = emp_id;

    IF empleado_existe = 0 THEN
        SET mensaje = 'Error: El empleado no existe';
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Empleado no encontrado';
    END IF;

    -- Verificar porcentaje válido
    IF porcentaje_aumento <= 0 OR porcentaje_aumento > 100 THEN
        SET mensaje = 'Error: Porcentaje debe estar entre 0 y 100';
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Porcentaje inválido';
    END IF;

    -- Obtener salario actual
    SELECT salario INTO salario_actual
    FROM empleados WHERE id = emp_id;

    -- Actualizar salario
    UPDATE empleados
    SET salario = salario * (1 + porcentaje_aumento / 100)
```

```
WHERE id = emp_id;

SET mensaje = CONCAT('Salario actualizado de ', salario_actual,
                    ' a ', salario_actual * (1 + porcentaje_aumento / 100));

END$$

DELIMITER ;

-- Uso del procedimiento
CALL ActualizarSalarioEmpleado(123, 10.5, @resultado);
SELECT @resultado;
```

Triggers (Disparadores)

Sintaxis Básica

```
DELIMITER $$

CREATE TRIGGER nombre_trigger
    [BEFORE|AFTER] [INSERT|UPDATE|DELETE]
    ON tabla
    FOR EACH ROW
BEGIN
    -- Lógica del trigger
END$$

DELIMITER ;
```

Cuándo se Ejecutan

- BEFORE - Antes de la operación (puede modificar NEW)
- AFTER - Después de la operación (no puede modificar NEW)

Valores OLD y NEW

- OLD.campo - Valor anterior (disponible en UPDATE y DELETE)
- NEW.campo - Valor nuevo (disponible en INSERT y UPDATE)

Disponibilidad por operación:

Operación	OLD	NEW
INSERT	✗	✓
UPDATE	✓	✓
DELETE	✓	✗

Ejemplos de Triggers

Trigger de Auditoría (AFTER DELETE):

```
DELIMITER $$
```

```
CREATE TRIGGER auditoria_empleados_delete
AFTER DELETE ON empleados
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_empleados (
        empleado_id,
        nombre_empleado,
        accion,
        fecha_accion,
        usuario
    ) VALUES (
        OLD.id,
        OLD.nombre,
        'DELETE',
        NOW(),
        USER()
    );
END$$
```

```
DELIMITER ;
```

Trigger de Validación (BEFORE INSERT):

```
DELIMITER $$
```

```
CREATE TRIGGER validar_empleado_insert
BEFORE INSERT ON empleados
FOR EACH ROW
BEGIN
    -- Validar edad mínima
```

```
IF DATEDIFF(CURDATE(), NEW.fecha_nacimiento) / 365 < 18 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'El empleado debe ser mayor de 18 años';
END IF;

-- Convertir nombre a mayúsculas
SET NEW.nombre = UPPER(NEW.nombre);

-- Asignar fecha de creación
SET NEW.fecha_creacion = NOW();
END$$

DELIMITER ;
```

Trigger de Actualización Automática (AFTER UPDATE):

```
DELIMITER $$

CREATE TRIGGER actualizar_stock_producto
AFTER UPDATE ON detalles_pedido
FOR EACH ROW
BEGIN
    -- Solo si cambió la cantidad
    IF NEW.cantidad != OLD.cantidad THEN
        UPDATE productos
        SET stock = stock - (NEW.cantidad - OLD.cantidad)
        WHERE id = NEW.producto_id;
    END IF;
END$$

DELIMITER ;
```



17 Funciones de Fecha y Hora

Obtener Fecha/Hora Actual

- NOW() - Fecha y hora actual (YYYY-MM-DD HH:MM:SS)
- CURDATE() - Fecha actual (YYYY-MM-DD)
- CURTIME() - Hora actual (HH:MM:SS)

Manipular Fechas

-- Sumar/restar tiempo

DATE_ADD(fecha, INTERVAL valor UNIDAD)

DATE_SUB(fecha, INTERVAL valor UNIDAD)

-- Ejemplos

DATE_ADD(CURDATE(), INTERVAL 15 DAY)

-- Hoy + 15 días

DATE_ADD(NOW(), INTERVAL 2 HOUR)

-- Ahora + 2 horas

DATE_SUB(CURDATE(), INTERVAL 1 MONTH)

-- Hoy - 1 mes

Unidades disponibles:

- DAY , WEEK , MONTH , YEAR
- HOUR , MINUTE , SECOND
- DAY_HOUR , DAY_MINUTE , YEAR_MONTH

Calcular Diferencias

DATEDIFF(fecha1, fecha2) -- Diferencia en días

TIMESTAMPDIFF(UNIDAD, fecha1, fecha2) -- Diferencia en cualquier unidad

-- Ejemplos

DATEDIFF(CURDATE(), '2023-01-01') -- Días desde inicio de año

TIMESTAMPDIFF(YEAR, fecha_nacimiento, CURDATE()) -- Edad en años

TIMESTAMPDIFF(MONTH, fecha_inicio, fecha_fin) -- Meses entre fechas

Extraer Partes de Fecha

YEAR(fecha) -- Año (2024)

MONTH(fecha) -- Mes (1-12)

DAY(fecha) -- Día (1-31)

HOUR(fecha) -- Hora (0-23)

MINUTE(fecha) -- Minuto (0-59)

SECOND(fecha) -- Segundo (0-59)

DAYOFWEEK(fecha) -- Día de semana (1=Domingo, 7=Sábado)

DAYNAME(fecha) -- Nombre del día ('Monday')

MONTHNAME(fecha) -- Nombre del mes ('January')

Formatear Fechas

DATE_FORMAT(fecha, formato)

-- Formatos comunes

DATE_FORMAT(NOW(), '%d/%m/%Y') -- 31/12/2024

DATE_FORMAT(NOW(), '%Y-%m-%d %H:%i:%s') -- 2024-12-31 23:59:59

DATE_FORMAT(NOW(), '%W, %M %D, %Y') -- Tuesday, December 31st, 2024

Funciones de Cadenas

Manipulación Básica

CONCAT(str1, str2, ...) -- Concatenar cadenas
CONCAT_WS(sep, str1, str2) -- Concatenar con separador
LENGTH(str) -- Longitud de cadena
CHAR_LENGTH(str) -- Longitud en caracteres
UPPER(str) / UCASE(str) -- Convertir a mayúsculas
LOWER(str) / LCASE(str) -- Convertir a minúsculas

Extracción y Búsqueda

SUBSTRING(str, pos, len) -- Extraer subcadena
LEFT(str, len) -- Primeros 'len' caracteres
RIGHT(str, len) -- Últimos 'len' caracteres
LOCATE(substr, str) -- Posición de subcadena (0 si no existe)
INSTR(str, substr) -- Similar a LOCATE

Limpieza de Cadenas

TRIM(str) -- Eliminar espacios al inicio y final
LTRIM(str) -- Eliminar espacios al inicio
RTRIM(str) -- Eliminar espacios al final
REPLACE(str, old, new) -- Reemplazar texto

Ejemplos Prácticos

-- Crear nombre completo

```

SELECT CONCAT(nombre, ' ', apellido) AS nombre_completo FROM empleados;

-- Crear iniciales
SELECT CONCAT(LEFT(nombre, 1), LEFT(apellido, 1)) AS iniciales FROM empleados;

-- Limpiar y formatear datos
UPDATE clientes
SET telefono = REPLACE(REPLACE(telefono, '-', ''), ' ', '')
WHERE telefono IS NOT NULL;

-- Extraer dominio de email
SELECT SUBSTRING(email, LOCATE('@', email) + 1) AS dominio FROM usuarios;

```



Funciones Matemáticas

Funciones Básicas

```

ABS(numero)           -- Valor absoluto
ROUND(numero, decimales) -- Redondear
CEILING(numero)       -- Redondear hacia arriba
FLOOR(numero)         -- Redondear hacia abajo
MOD(dividendo, divisor) -- Resto de división
POWER(base, exponente) -- Potencia
SQRT(numero)          -- Raíz cuadrada

```

Funciones de Agregación Matemática

```

SUM(campo)           -- Suma
AVG(campo)           -- Promedio
MAX(campo)           -- Máximo
MIN(campo)           -- Mínimo
COUNT(campo)        -- Contar (excluyendo NULL)
COUNT(*)            -- Contar todas las filas

```



Funciones Condicionales

CASE - Condicional Múltiple

```
-- Sintaxis simple
CASE campo
  WHEN valor1 THEN resultado1
  WHEN valor2 THEN resultado2
  ELSE resultado_por_defecto
END

-- Sintaxis con condiciones
CASE
  WHEN condicion1 THEN resultado1
  WHEN condicion2 THEN resultado2
  ELSE resultado_por_defecto
END
```

Ejemplo:

```
SELECT
  nombre,
  salario,
  CASE
    WHEN salario < 2000 THEN 'Bajo'
    WHEN salario BETWEEN 2000 AND 4000 THEN 'Medio'
    ELSE 'Alto'
  END AS categoria_salario
FROM empleados;
```

IF - Condicional Simple

IF(condicion, valor_si_true, valor_si_false)

```
-- Ejemplo
SELECT nombre, IF(activo = 1, 'Activo', 'Inactivo') AS estado
FROM empleados;
```

IFNULL y COALESCE - Manejar Valores NULL

IFNULL(campo, valor_por_defecto) -- Si campo es NULL, usar valor_por_defecto
COALESCE(campo1, campo2, campo3) -- Primer valor no NULL

```
-- Ejemplos
SELECT nombre, IFNULL(telefono, 'Sin teléfono') FROM clientes;
```

```
SELECT COALESCE(telefono_movil, telefono_fijo, 'Sin contacto') FROM clientes;
```

Estados SQLSTATE Comunes

Para uso en procedimientos almacenados y manejo de errores:

- '00000' - Éxito
- '45000' - Error definido por usuario (más común)
- '23000' - Violación de restricción de integridad
- '42000' - Error de sintaxis o violación de acceso
- 'HY000' - Error general

Ejemplo de uso:

```
IF condicion_error THEN  
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Descripción del error';  
END IF;
```

Consejos y Buenas Prácticas

Para Consultas:

1. **Siempre usar alias** en consultas con múltiples tablas
2. **Especificar campos** en lugar de usar `SELECT *`
3. **Usar índices** en campos de WHERE y JOIN
4. **Limitar resultados** con LIMIT cuando sea apropiado

Para Modificaciones:

1. **Siempre hacer backup** antes de UPDATE/DELETE masivos
2. **Usar transacciones** para operaciones críticas
3. **Verificar con SELECT** antes de ejecutar UPDATE/DELETE
4. **Usar WHERE** siempre en UPDATE y DELETE

Para Procedimientos:

1. **Validar parámetros** al inicio
2. **Usar nombres descriptivos** para variables
3. **Manejar errores** con SIGNAL SQLSTATE
4. **Documentar** la lógica con comentarios

Para Triggers:

1. **Mantener lógica simple** y eficiente
2. **Evitar triggers** que modifiquen la misma tabla
3. **Considerar el rendimiento** especialmente en BEFORE triggers
4. **Documentar** qué hace cada trigger

Optimización:

1. **Crear índices** en campos consultados frecuentemente
2. **Evitar subconsultas** cuando sea posible, usar JOIN
3. **Usar LIMIT** en consultas de prueba
4. **Analizar planes de ejecución** con EXPLAIN